Academic Year: 2026-2027
Semester: III
Class / Branch: SE IT
Subject:ADSAL lab
Name of Instructor:prof.Sneha Dalvi

Name of Student:Nishant Suryarao
Student ID:26204001
Date of Performance:06/10/2026
Date of Submission:06/10/2026

Experiment No.08

Aim: Implement Sum of subsets problem for a given set of distinct numbers using backtracking.

Code:

```
#include <stdio.h>
int a[] = {2, 3, 5, 7}, n = 4, target = 10;
int x[4]; void subset(int i, int sum) { if
(sum == target) { for (int j = 0; j < i; j++)
if (x[j]) printf("%d ", a[j]); printf("\n");
return;
}
if (i == n || sum >
target) return; x[i] = 1;
subset(i + 1, sum + a[i]);
x[i] = 0; subset(i + 1,
sum);
}
int main() {
subset(0, 0);
return 0;
}
```

Output Screen Shots:

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ gcc -o Exp8 Exp8.c
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ ./Exp8
2 3 5
3 7
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$
```

Conclusion : The Sum of Subsets problem was successfully implemented using backtracking. The algorithm explored the inclusion and exclusion of each element and found all subsets whose sum matched the given target.

Experiment 8: Sum of Subsets Using Backtracking

Definition: The Sum of Subsets problem finds all subsets of a given set whose elements add up to a specified target value.

Algorithm:

1. Start with the given set and target sum.
2. Initialize the current sum to zero.
3. Consider the first element.
4. Include the element and recursively explore the next element.
5. Backtrack and explore the case where the element is excluded.
6. If the current sum equals the target, display the subset.
7. If all elements are considered or the sum exceeds the target, stop exploring that branch.
8. Repeat for all possible subsets.
9. Stop.

Conclusion: Thus, the Sum of Subsets problem was successfully implemented using backtracking to find subsets whose sum equals the target.

Example from your PDF: Set = {2, 3, 5, 7}, target = 10. The subsets are {3, 7} and {2, 3, 5}.

Complexity: Worst-case time \(O(2^n)\).